L20 - 推理优化

> **"又快又好地生成"**

本节目标

1. 理解自回归生成的瓶颈
2. 掌握 KV-Cache 的原理与实现
3. 理解 YaRN 长度外推技术
4. 掌握各种生成策略：Greedy、Top-K、Top-P、Temperature
5. 了解 FlashAttention 和模型量化的基本概念

前置知识

- L04 Attention 机制的 Q/K/V 计算
- L05 RoPE 位置编码
- L06 完整 Transformer 前向传播流程
- 概率分布、采样的基本概念

一、自回归生成的瓶颈

1.1 自回归生成是怎么工作的

LLM 生成文本是一个 **token-by-token** 的过程：

```
Step 1: "今天"          → 模型预测下一个 token → "天气"
Step 2: "今天天气"       → 模型预测下一个 token → "真"
Step 3: "今天天气真"     → 模型预测下一个 token → "好"
...
```

每生成一个 token，模型都需要做一次**完整的前向传播**（从 Embedding 到最后的 LM Head）。

1.2 瓶颈分析

假设要生成 100 个 token：

朴素方式（无优化）：

```
Step 1: 计算 "今天" 的 Q, K, V → Attention → FFN → 输出 "天气"
Step 2: 计算 "今天天气" 的 Q, K, V → Attention → FFN → 输出 "真"
Step 3: 计算 "今天天气真" 的 Q, K, V → Attention → FFN → 输出 "好"
```

在 Step 3 中，"今天" 和 "天气" 的 K、V 在 Step 1 和 Step 2 中已经算过了，但朴素方式又重新算了一遍！

重复计算量：生成第 n 个 token 时，前 $n-1$ 个 token 的 K/V 被重复计算。总的重复计算量与 n^2 成正比。

二、KV-Cache 详解

2.1 核心思想

KV-Cache 的思想非常简单：**把已经计算过的 K 和 V 缓存起来，下次直接用**。

```
Step 1: 计算 K₁, V₁（"今天"）→ 缓存 [K₁], [V₁]
Step 2: 计算 K₂, V₂（"天气"）→ 缓存 [K₁,K₂], [V₁,V₂]
Step 3: 只计算 K₃, V₃（"真"）→ 缓存 [K₁,K₂,K₃], [V₁,V₂,V₃]
```

每次前向传播时： - **Q**：只计算当前新 token 的 Query - **K, V**：只计算当前新 token 的 Key 和 Value，然后和缓存的历史 K/V 拼接

2.2 为什么只缓存 K 和 V，不缓存 Q?

在 Attention 计算中：

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{QK^T}{\sqrt{d_k}})V$$

- **Q** 是当前 token 的查询，每次生成的 token 不同，Q 必须重新计算
- **K** 是所有历史 token 的键，历史部分不会变，可以缓存
- **V** 是所有历史 token 的值，同理可以缓存

2.3 KV-Cache 的实现

MiniMind 中的 KV-Cache 实现核心逻辑：

```
class Attention(nn.Module):
    def forward(self, x, pos, kv_cache=None):
        B, T, D = x.shape

        # 计算当前 token 的 Q, K, V
        q = self.wq(x)  # [B, T, D]
        k = self.wk(x)  # [B, T, D_kv]
        v = self.wv(x)  # [B, T, D_kv]

        # 应用 RoPE
        q, k = apply_rope(q, k, pos)

        if kv_cache is not None:
            # 拼接历史缓存
            cached_k, cached_v = kv_cache
            k = torch.cat([cached_k, k], dim=1)  # [B, T_total, D_kv]
            v = torch.cat([cached_v, v], dim=1)  # [B, T_total, D_kv]

        # 更新缓存
        new_kv_cache = (k, v)

        # GQA: 扩展 K/V 的 head 数量
        k = repeat_kv(k, self.n_rep)
        v = repeat_kv(v, self.n_rep)

        # Attention 计算
        # Q 只有当前 token: [B, n_heads, 1, d_head]
        # K/V 包含所有历史: [B, n_heads, T_total, d_head]
        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.d_head)
        attn = F.softmax(scores, dim=-1)
        output = torch.matmul(attn, v)

        return output, new_kv_cache
```

2.4 训练时不需要 KV-Cache

训练时：整个序列一次性输入，所有 token 并行计算 Attention，不存在重复计算的问题。因此训练时**不需要**也**不应该**使用 KV-Cache。

推理时：逐 token 生成，需要用 KV-Cache 避免重复计算。

这是一个常见的面试考点！

2.5 KV-Cache 的内存开销

KV-Cache 用空间换时间，内存开销计算如下：

```
每层 KV-Cache 大小 = 2 × batch_size × seq_len × kv_heads × head_dim × dtype_size

对于 MiniMind (假设):
  - 层数: 8
  - kv_heads: 2 (GQA)
  - head_dim: 64
  - seq_len: 32768
  - batch_size: 1
  - dtype: float16 (2 bytes)

  每层: 2 × 1 × 32768 × 2 × 64 × 2 = 16MB
  总共: 8 × 16MB = 128MB
```

对于大模型（如 70B），KV-Cache 可能占用数十 GB 的显存，成为推理的显存瓶颈。

2.6 KV-Cache 的优化技术

- **GQA（Grouped Query Attention）**：减少 KV heads 数量，直接减小 KV-Cache
- **MQA（Multi-Query Attention）**：极端情况，只有 1 个 KV head
- **Paged KV-Cache**（vLLM 使用）：像操作系统的虚拟内存一样管理 KV-Cache
- **量化 KV-Cache**：用 INT8/INT4 存储 K/V

三、YaRN 长度外推

3.1 问题背景

MiniMind 训练时的最大序列长度（max_pos）是 32768。但如果用户输入了更长的文本怎么办?

朴素的 RoPE 不支持外推：如果位置编号超过训练时见过的范围，RoPE 的旋转角度会出现训练时没见过的模式，导致 Attention 分数紊乱，输出质量急剧下降。

3.2 YaRN 的基本思想

YaRN（Yet another RoPE extensioN）是一种 RoPE 的长度外推方法，核心思想是**对旋转频率进行缩放**，使得更长的序列也能映射到训练时见过的旋转角度范围。

传统 RoPE 的频率：

$\theta_i = 10000^{-2i/d}$

YaRN 对频率做缩放：

$$\theta_i^{'} = \frac{\theta_i}{s(i)}$$

其中 $s(i)$ 是一个与维度位置 i 相关的缩放因子。YaRN 的关键贡献是将频率分为三个区域：

1. **高频分量**（变化快的维度）：不缩放（保持局部位置精度）
2. **低频分量**（变化慢的维度）：线性缩放（扩展长距离位置感知）
3. **中间频率**：平滑插值过渡

3.3 缩放效果

MiniMind 支持通过 YaRN 在推理时将上下文长度从 32768 扩展到：

$32768 \times 16 = 524288 \approx 512K$ tokens

这意味着推理时可以处理远超训练长度的输入，虽然效果可能不如在该长度上训练过的模型。

3.4 MiniMind 中的 YaRN 实现

```
def precompute_freqs_cis(dim, end, theta=10000.0, scaling_factor=1.0):
    """预计算 RoPE 的频率，支持 YaRN 缩放"""
    freqs = 1.0 / (theta ** (torch.arange(0, dim, 2).float() / dim))

    if scaling_factor != 1.0:
        # YaRN 风格的频率缩放
        low_freq_factor = 1.0
        high_freq_factor = 4.0
        old_context_len = end

        low_freq_wavelen = old_context_len / low_freq_factor
        high_freq_wavelen = old_context_len / high_freq_factor

        new_freqs = []
        for freq in freqs:
            wavelen = 2 * math.pi / freq
            if wavelen < high_freq_wavelen:
                # 高频：不缩放
                new_freqs.append(freq)
            elif wavelen > low_freq_wavelen:
                # 低频：线性缩放
                new_freqs.append(freq / scaling_factor)
            else:
                # 中间频率：插值
                smooth = (old_context_len / wavelen - low_freq_factor) / \
                         (high_freq_factor - low_freq_factor)
                new_freqs.append((1 - smooth) * freq / scaling_factor + smooth * freq)
        freqs = torch.tensor(new_freqs)

    t = torch.arange(end)
    freqs = torch.outer(t, freqs)
    return torch.polar(torch.ones_like(freqs), freqs)
```

四、生成策略

4.1 贪心搜索（Greedy Search）

每一步选择概率最高的 token：

```
def greedy_search(logits):
    return logits.argmax(dim=-1)
```

特点：确定性输出，速度最快，但容易陷入重复、缺乏多样性。

4.2 Temperature 参数

Temperature 控制输出概率分布的"锐利程度"：

$$p_i = \frac{\exp(z_i / T)}{\sum_j \exp(z_j / T)}$$

- **T < 1**：分布更"尖锐"，高概率 token 更突出，输出更确定
- **T = 1**：原始分布
- **T > 1**：分布更"平坦"，低概率 token 有更多机会，输出更多样

```
def apply_temperature(logits, temperature=1.0):
    return logits / temperature
```

直觉理解：Temperature 就像调节"创造力"的旋钮。T=0.1 写公文，T=1.0 聊天，T=1.5 写诗。

4.3 Top-K 采样

只从概率最高的 K 个 token 中采样：

```
def top_k_sampling(logits, k=50):
    values, indices = torch.topk(logits, k)
    # 将非 Top-K 的 token 概率设为 -inf
    logits[logits < values[:, -1:]] = float('-inf')
    probs = F.softmax(logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)
```

问题：K 是固定的。有时候模型非常确定下一个 token（只有 2-3 个合理选择），K=50 会引入太多噪声；有时候模型不确定（10-20 个 token 都合理），K=50 可能仍然不够。

4.4 Top-P（Nucleus Sampling）

动态选择最小的 token 集合，使得累积概率达到 P：

```
def top_p_sampling(logits, p=0.9):
    sorted_logits, sorted_indices = torch.sort(logits, descending=True)
    cumulative_probs = torch.cumsum(F.softmax(sorted_logits, dim=-1), dim=-1)

    # 找到累积概率超过 p 的位置
    sorted_indices_to_remove = cumulative_probs > p
    # 保留第一个超过 p 的 token（确保至少有一个）
    sorted_indices_to_remove[:, 1:] = sorted_indices_to_remove[:, :-1].clone()
    sorted_indices_to_remove[:, 0] = False

    # 将被移除的 token 概率设为 -inf
    indices_to_remove = sorted_indices_to_remove.scatter(
        1, sorted_indices, sorted_indices_to_remove
    )
    logits[indices_to_remove] = float('-inf')

    probs = F.softmax(logits, dim=-1)
    return torch.multinomial(probs, num_samples=1)
```

优势：自适应——当模型确定时只选少数 token，不确定时选更多 token。

4.5 Top-K vs Top-P

维度	Top-K	Top-P
候选数量	固定 K 个	动态，取决于概率分布
适应性	差（K 不变）	好（自适应分布形状）
参数	K（通常 50）	P（通常 0.9-0.95）
适用场景	简单场景	更通用

实践中常组合使用：先 Top-K 过滤，再 Top-P 过滤，最后用 Temperature 调节。

4.6 Repetition Penalty（重复惩罚）

降低已经生成过的 token 的概率，防止"车轱辘话来回说"：

```
def apply_repetition_penalty(logits, generated_ids, penalty=1.2):
    for token_id in set(generated_ids):
        if logits[token_id] > 0:
            logits[token_id] /= penalty
        else:
```

```
            logits[token_id] *= penalty
    return logits
```

penalty > 1 时会降低已生成 token 的概率。典型值 1.1-1.3。

4.7 生成策略的完整 Pipeline

```
def generate_token(model, input_ids, kv_cache, config):
    # 1. 模型前向传播
    logits, kv_cache = model(input_ids[:, -1:], kv_cache=kv_cache)
    logits = logits[:, -1, :]  # 取最后一个位置的 logits

    # 2. Repetition Penalty
    logits = apply_repetition_penalty(logits, input_ids[0], config.rep_penalty)

    # 3. Temperature
    logits = logits / config.temperature

    # 4. Top-K
    if config.top_k > 0:
        logits = top_k_filter(logits, config.top_k)

    # 5. Top-P
    if config.top_p < 1.0:
        logits = top_p_filter(logits, config.top_p)

    # 6. 采样
    probs = F.softmax(logits, dim=-1)
    next_token = torch.multinomial(probs, num_samples=1)

    return next_token, kv_cache
```

五、FlashAttention 简介

5.1 标准 Attention 的内存问题

标准 Attention 需要先计算完整的 QK^T 矩阵（大小 $T \times T$），再做 Softmax，再乘 V。对于长序列，这个 $T \times T$ 矩阵可能非常大。

5.2 FlashAttention 的核心思想

FlashAttention 通过**分块计算（tiling）**来避免存储完整的 Attention 矩阵：

1. 将 Q、K、V 切成小块
2. 在 GPU 的高速 SRAM 中计算小块的 Attention
3. 用在线 Softmax 算法逐块累积结果

4. 不需要在 HBM 中存储完整的 $T \times T$ 矩阵

效果： - 内存从 $O(T^2)$ 降到 $O(T)$ - 速度提升 2-4 倍（减少 HBM 读写）

5.3 在 MiniMind 中使用

```
# PyTorch 2.0+ 内置了 FlashAttention 支持
output = F.scaled_dot_product_attention(
    q, k, v,
    attn_mask=mask,
    is_causal=True  # 自动使用因果 mask + FlashAttention
)
```

六、模型量化简介

6.1 为什么需要量化

模型参数通常用 FP32（4 bytes）或 FP16（2 bytes）存储。量化就是用更少的位数表示：

```
FP32: 4 bytes/param → 64M params = 256MB
FP16: 2 bytes/param → 64M params = 128MB
INT8: 1 byte/param  → 64M params = 64MB
INT4: 0.5 byte/param → 64M params = 32MB
```

6.2 量化类型

- **权重量化（Weight Quantization）**：只量化模型权重，推理时反量化回 FP16 计算
- **激活量化（Activation Quantization）**：同时量化权重和中间激活
- **KV-Cache 量化**：量化 KV-Cache 以节省显存

6.3 常见量化方法

- **GPTQ**：基于 Hessian 矩阵的逐层量化
- **AWQ**：保护重要权重通道的量化
- **GGUF**：llama.cpp 使用的量化格式，支持 Q4_K_M、Q5_K_M 等

6.4 量化与 MiniMind

MiniMind 支持转换为 GGUF 格式，使用 llama.cpp 推理时可以选择不同的量化级别。对于 64M 的小模型，量化的收益相对有限（模型本身就很小），但对于学习量化概念很有价值。

🎤 面试考点

Q1: 什么是 KV-Cache？为什么需要它？

答：KV-Cache 是在自回归推理时缓存历史 token 的 Key 和 Value 向量的技术。因为自回归生成时，每生成一个新 token 都需要和所有历史 token 做 Attention，如果不缓存 K/V，历史 token 的 K/V 会被反复计算。KV-Cache 通过空间换时间，将推理复杂度从 $O(n^2)$ 降低到 $O(n)$（针对单步生成）。

Q2: 为什么训练时不使用 KV-Cache？

答：训练时整个序列一次性输入模型，所有 token 的 Q/K/V 通过矩阵运算并行计算，不存在重复计算问题。KV-Cache 只在逐 token 的自回归推理时才有意义。另外，训练时使用 Teacher Forcing，每个位置的目标是已知的，不需要真正地自回归生成。

Q3: KV-Cache 的内存开销怎么计算？

答：每层 KV-Cache 大小 = 2（K 和 V）× batch_size × seq_len × kv_heads × head_dim × dtype_bytes。总开销 = 层数 × 每层大小。对于大模型和长序列，KV-Cache 可能成为推理显存的主要瓶颈。

Q4: Top-P 和 Top-K 采样的区别？各有什么优劣？

答： - Top-K：固定选择概率最高的 K 个 token，不自适应分布形状 - Top-P：动态选择累积概率达到 P 的最小 token 集合，自适应概率分布 - Top-P 更灵活——当模型确定时自动缩小候选集，不确定时扩大候选集 - 实践中常组合使用

Q5: Temperature 参数的作用是什么？

答：Temperature 缩放 logits 后再做 Softmax。T<1 使分布更尖锐（高概率 token 更突出，输出更确定性），T>1 使分布更平坦（低概率 token 有更多机会，输出更多样）。T→0 等价于贪心搜索，T→∞ 等价于均匀随机采样。

Q6: YaRN 是什么？它解决什么问题？

答：YaRN 是一种 RoPE 位置编码的长度外推方法。它解决的问题是：模型训练时的最大序列长度有限，但推理时可能需要处理更长的文本。YaRN 通过对 RoPE 频率进行分区缩放（高频不动、低频缩放、中间插值），使模型能够泛化到更长的位置。

Q7: FlashAttention 的核心原理是什么？

答：FlashAttention 通过分块计算（tiling）避免在 GPU HBM 中存储完整的 $T \times T$ Attention 矩阵。它利用 GPU SRAM 的高带宽来计算小块 Attention，用在线 Softmax 逐块累积结果，将内存复杂度从 $O(T^2)$ 降到 $O(T)$，同时减少 HBM 读写次数提升速度。

✅ 自测题

1. **填空**：KV-Cache 缓存的是 _ ***和*** _，不缓存 ___。
2. **判断**：训练时也需要使用 KV-Cache 来加速。（对/错）
3. **选择**：Temperature=0.1 时，模型的输出会：
4. A. 更加随机多样
5. B. 更加确定性
6. C. 完全均匀随机
7. D. 无影响
8. **简答**：为什么 Top-P 比 Top-K 更灵活？举例说明。
9. **计算**：一个 32 层模型，hidden_dim=4096，kv_heads=8，head_dim=128，batch_size=1，seq_len=4096，用 FP16 存储。请计算 KV-Cache 的总大小。

▼ 参考答案

1. Key 和 Value，不缓存 Query。 2. 错。训练时整个序列并行输入，不需要 KV-Cache。 3. B。T<1 使分布更尖锐，输出更确定。 4. 假设词表中 token A 概率 0.9，其余各 0.001。Top-K=50 会选 50 个 token，引入了很多不合理的选择。Top-P=0.95 只需要选 token A 就够了（0.9<0.95，最多再加一个），自适应地给出更合理的候选集。 5. 每层 KV-Cache = 2 × 1 × 4096 × 8 × 128 × 2 bytes = 16,777,216 bytes = 16 MB 总共 = 32 × 16 MB = 512 MB

下一节预告

下一节我们将学习 **部署与应用**，了解如何将训练好的 MiniMind 模型通过 ollama、vLLM、llama.cpp 等框架部署为可用的推理服务，以及 Tool Calling 和 Adaptive Thinking 等高级功能。